WinCUPL II

WinCUPL II User Guide

Product Page Links

Summary

WinCUPL II is Microchip's updated, free, Windows-based development environment for designing and deploying logic to programmable logic devices using the CUPL hardware description language. It offers a clean and simple workflow, from design entry through simulation and JEDEC® programming file generation, and it supports Microchip's full line of SPLDs and CPLDs.

Features

- Hardware Description Language: Use the powerful and easy-to-learn CUPL language created for SPLDs and CPLDs to describe complex logic with simple Boolean equations, truth tables and state machines.
- Integrated Development Environment: Seamlessly write, compile and debug your designs without leaving the application.
- Built-in Functional Simulator: Test and verify your logic with the included functional simulator before programming your PLD, saving time and preventing errors.
- Wide Device Support: Can be used with all Microchip SPLDs and CPLDs.
- Industry-Standard Output: Generate JEDEC programming files that are compatible with virtually all Microchip and third-party device programmers.

© 2026 Microchip Technology Inc. and its subsidiaries

Table of Contents

Summary....1

Features....1

1. Introduction to Programmable Logic....3
   - 1.1. What Is Programmable Logic?....3
   - 1.2. Device Technologies....7
   - 1.3. Programming Programmable Logic Devices....8
   - 1.4. Functionally Testing Logic Devices....8

2. Designing with CUPL Language....9
   - 2.1. Declaration of Language Elements....9
   - 2.2. Usage of the Language Syntax....10
   - 2.3. Advanced Language Syntax....20

3. Using WinCUPL II....24
   - 3.1. Overview....24
   - 3.2. Data and Design Flows....25

4. WinCUPL II Tutorial....28
   - 4.1. 2-Input AND Gate....28

5. Revision History....36

Microchip Information....37
- Trademarks....37
- Legal Notice....37
- Microchip Devices Code Protection Feature....37

Product Page Links....38

© 2026 Microchip Technology Inc. and its subsidiaries

1. Introduction to Programmable Logic

1.1. What Is Programmable Logic?

Programmable logic, as the name implies, is a family of components that contains arrays of logic elements (AND, OR, INVERT, LATCH, FLIP-FLOP) that can be configured into any logical function that the user desires and the component supports. There are several classes of programmable logic devices: ASICs, FPGAs, PAL/GAL-based Simple PLDs (SPLDs), Complex PLDs (CPLDs) and PROMs.

1.1.1. ASICs

ASICs are *Application Specific Integrated Circuit* devices, and they are mentioned here because they are user-definable devices. Unlike other devices, ASICs may contain analog, digital or combinations of analog and digital functions. In general, they are mask-programmable and not user-programmable. This means that manufacturers configure the devices according to user specifications. ASICs are used to combine a large number of logic functions into a single device. However, these devices have a high initial cost; therefore, they are mainly used in high-volume applications. Due to the nature of ASICs, CUPL and other similar programmable logic languages cannot support these devices.

1.1.2. Basic Architecture of a User-Programmable Device

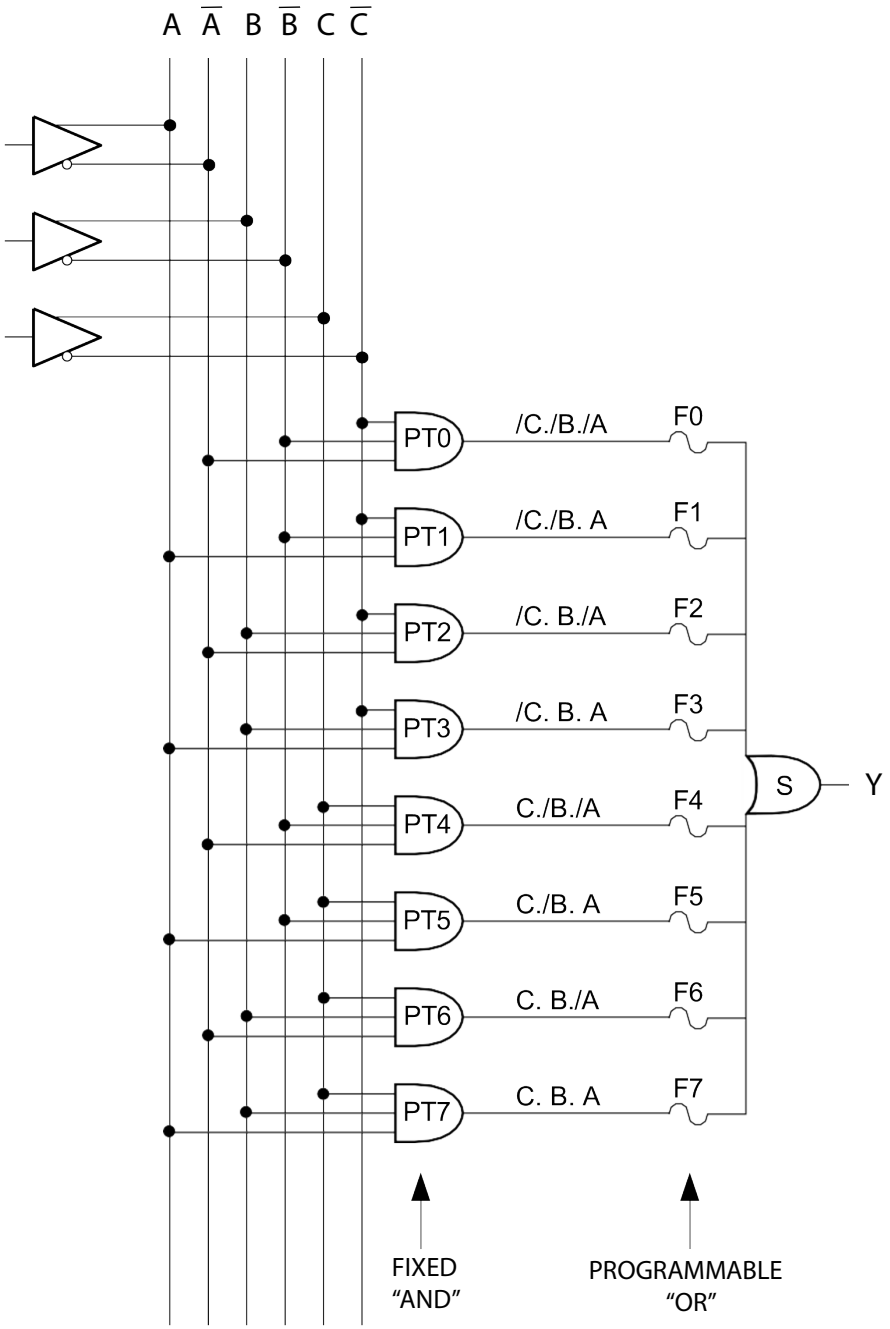
First, a *user-programmable device* is one that contains a predefined general architecture in which a user can program a design into the device using a set of development tools. The general architecture may vary but normally consists of one or more arrays of AND and OR terms for implementing logic functions. Many devices also contain combinations of flip-flops and latches, which may be used to implement sequential logic or to serve as storage elements for the inputs and outputs of a device. More complex devices contain *macrocells*. Macrocells allow the user to configure the types of inputs and outputs needed for a design.

1.1.3. PROMs

PROMs are *Programmable Read-Only Memory* devices. Even though the name does not imply programmable logic, PROMs are, in fact, devices that can be used to implement logic functions. The architecture of most PROMs typically consists of a fixed number of AND array terms that feed a programmable OR array. They are mainly used for decoding specific input combinations into output functions, such as memory mapping in microprocessor environments.

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 1-1. Elementary PROM Architecture



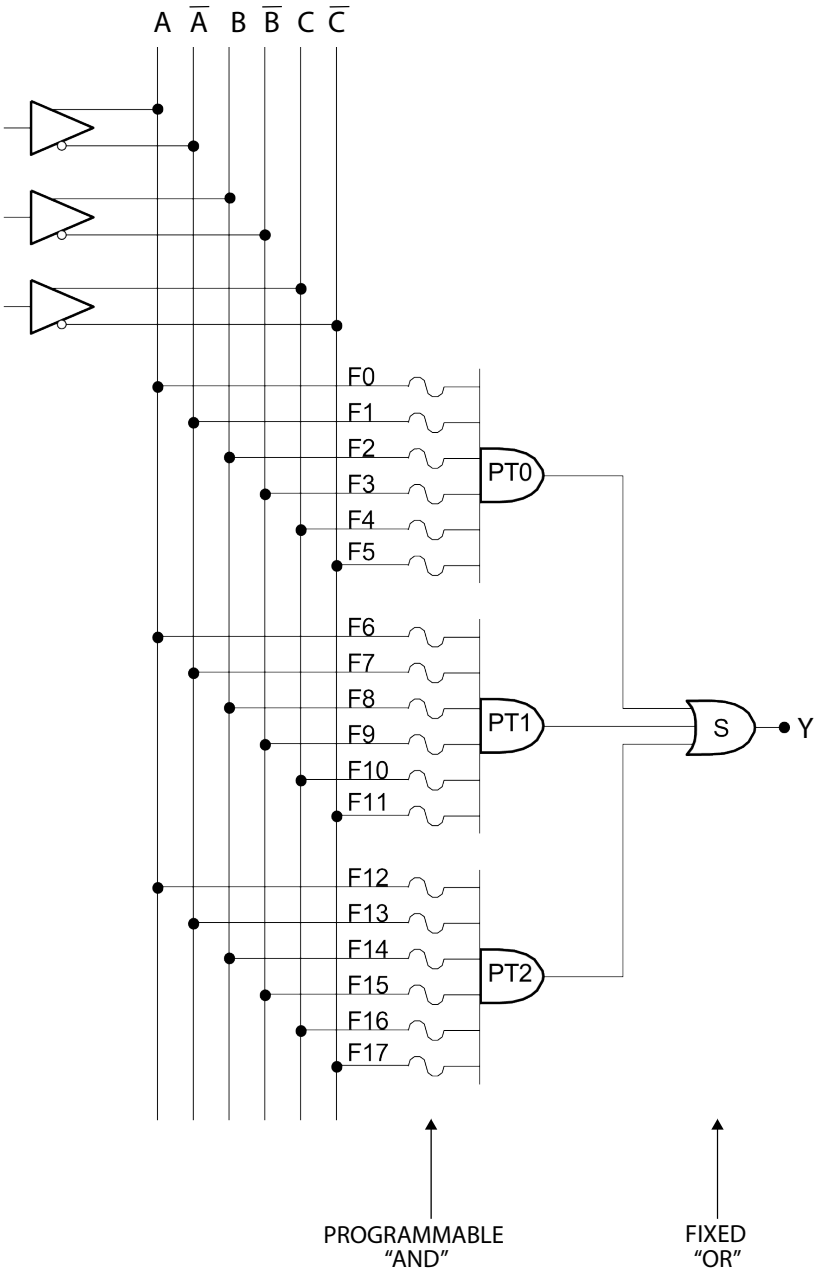
1.1.4. PALs

PALs are *Programmable Array Logic* devices. The internal architecture consists of programmable AND terms feeding fixed OR terms. All inputs to the array can be ANDed together, but specific AND terms are dedicated to specific OR terms. PALs have a very popular architecture and are possibly the most widely used type of user-programmable device. If a device contains macrocells, it will usually have a PAL architecture. Typical macrocells may be programmed as inputs, outputs, or input/output (I/O) using a tri-state enable. They normally have output registers that may or may not be used in conjunction with the associated I/O pin. Other macrocells have more than one register, various

© 2026 Microchip Technology Inc. and its subsidiaries

types of feedback into the arrays, and occasionally feedback between macrocells. These devices are mainly used to replace multiple discrete logic functions, commonly referred to as *glue logic*.

Figure 1-2. Elementary PAL Architecture



1.1.5. GALs

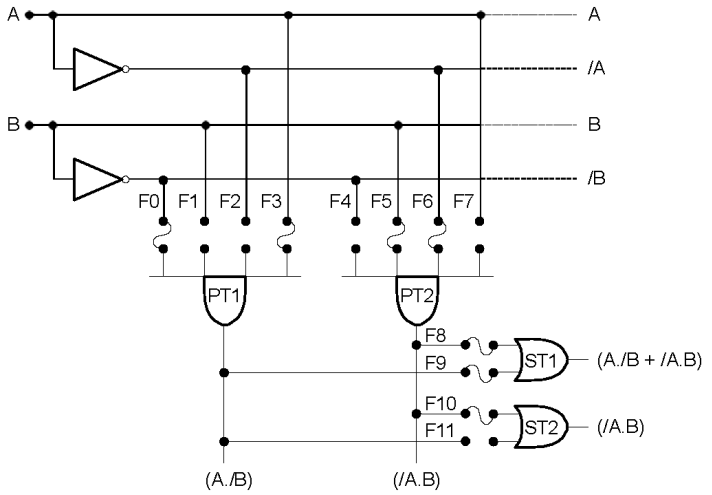
GALs are *Generic Array Logic* devices. They are designed to emulate many common PALs through the use of macrocells. This reduces the number of different devices in stock and increases the quantity purchased. Usually, purchasing a large quantity of the same device lowers the individual device cost. Additionally, these devices are electrically erasable, which makes them very useful for design engineers.

© 2026 Microchip Technology Inc. and its subsidiaries

1.1.6. PLAs

PLAs are *Programmable Logic Arrays*. These devices contain both programmable AND and OR terms, which allow any AND term to feed any OR term. As a result, PLAs have greater flexibility than PALs with regard to logic functionality. They typically have feedback from the OR array back into the AND array, which may be used to implement asynchronous state machines. Most state machines, however, are implemented as synchronous machines. With this in mind, manufacturers created a type of PLA called a *Sequencer*, which has registered feedback from the output of the OR array into the AND array.

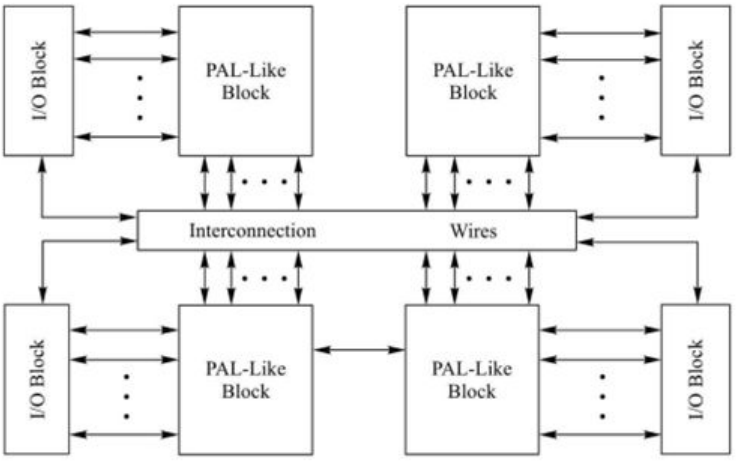
Figure 1-3. Elementary PLA Architecture



1.1.7. CPLDs

The basic architecture of a CPLD is very similar to that of a PAL SPLD, but it has a much higher number of macrocells, and the architecture of the macrocell is more complex. Another trait that most CPLDs have is a wide, centralized interconnect bus that connects the various logic elements to each other and to the I/O pins. These features allow complex designs to be implemented within these devices while maintaining predictable timing characteristics.

Figure 1-4. Elementary CPLD Architecture

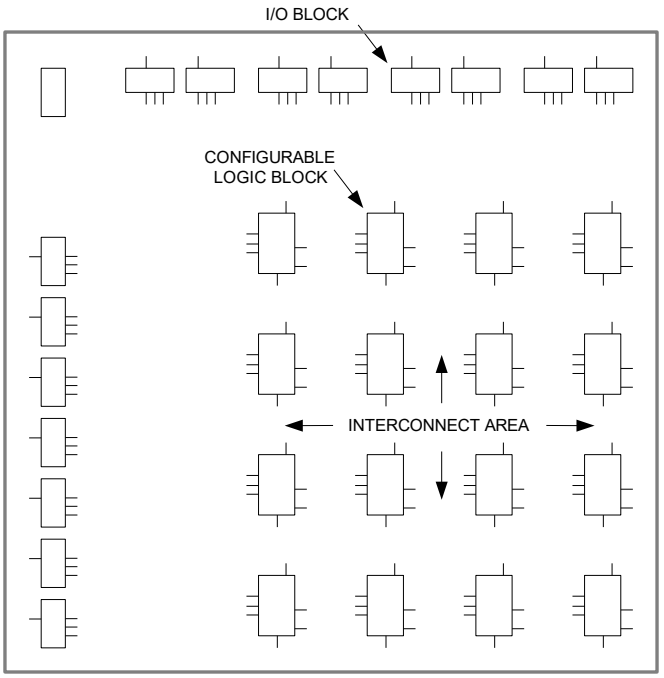


© 2026 Microchip Technology Inc. and its subsidiaries

1.1.8. FPGAs

FPGAs are *Field Programmable Gate Array* devices. Simply put, they are electrically programmable gate array ICs that contain multiple levels of logic. FPGAs offer high gate density and performance, along with a large number of user-definable inputs and outputs, a flexible interconnect scheme, and a gate-array-like design environment. They are not constrained to the typical AND-OR array. Instead, they contain an interior matrix of configurable logic blocks (CLBs) and a surrounding ring of I/O blocks (IOBs). Each CLB contains programmable combinatorial logic and storage registers. The combinatorial logic section of the block is capable of implementing any Boolean function of its input variables. Each IOB can be programmed independently to be an input, an output with tri-state control or a bi-directional pin. It also contains flip-flops that can be used to buffer inputs and outputs. The interconnection resources consist of a network of lines that run horizontally and vertically in the rows and columns between the CLBs. Programmable switches connect the inputs and outputs of IOBs and CLBs to nearby lines. Long lines run the entire length or breadth of the device, bypassing interchanges to provide distribution of critical signals with minimum delay or skew. Designers using FPGAs can define the logic functions of a circuit and revise these functions as necessary.

Figure 1-5. Elementary FPGA Architecture



1.2. Device Technologies

Some of the common technologies available are CMOS (Complementary Metal Oxide Semiconductor), bipolar TTL, GaAs (Gallium Arsenide), and ECL (Emitter Coupled Logic), as well as combination fabrications such as BiCMOS and ECL/bipolar. The faster semiconductor technologies, such as ECL and GaAs, are also more power-hungry. Generally, speed is proportional to power consumption.

© 2026 Microchip Technology Inc. and its subsidiaries

1.2.1. Device Packaging

Programmable devices come in many shapes and sizes. Most devices are available in the following physical configurations: DIP (Dual Inline Package), PLCC (Plastic Leaded Chip Carrier), QFP (Quad Flat Pack), BGA (Ball Grid Array), SOIC (Small Outline I.C.), TSOP (Thin Small Outline), PGA (Pin Grid Array), etc. These devices can be rectangular with pins on two sides, square with pins on all sides, or square with pins, pads or balls on the underside. It is important for the hardware and software development tools to fully support as many device types as possible to take full advantage of the myriad of devices on the market.







1.3. Programming Programmable Logic Devices

Programmable logic devices are programmed by either shorting or opening connections within a device array, thereby connecting or disconnecting inputs to a gate. Most hardware programmers receive a fuse information file from a software development package in ASCII text format. The ASCII file is typically in JEDEC format for PLDs or HEX format for PROMs. This file contains the information necessary for the programmer to program the device. The JEDEC file contains fuse connections that are represented by an address followed by a series of 1s and 0s, where a “1” indicates a disconnected state and a “0” indicates a connected state. The JEDEC file can also contain information that enables the hardware programmer to perform a functional test on the device.

1.4. Functionally Testing Logic Devices

A functional test may be performed after programming a device, provided that the hardware and software development package supports the generation and use of test vectors. Test vectors consist of a list of pins for the design, input values for each step of the functional test and a list of expected outputs from the circuit. The programmer sequences through the input values, checks for the predicted outputs and reports the results to the user. This process allows design engineers and production crews to verify that the programmed device works as designed.

© 2026 Microchip Technology Inc. and its subsidiaries

2. Designing with CUPL Language

When creating any design, it is generally considered good practice to implement it using a “top-down” approach. A top-down design is characterized by starting with a global definition of the design and then repeating the global definition process for each element of the main definition until the entire project has been defined. CUPL offers many features that accommodate this type of design. This section describes the instructions that CUPL provides for implementing a design.

2.1. Declaration of Language Elements

This section describes the elements that comprise the CUPL logic description language.

2.1.1. Pin/Node Definition

Since the pin definitions must be declared at the beginning of the source file, their definition is a natural starting point for a design. Nodes and pin nodes, which are used to define buried registers, should also be declared at the beginning of the source file. Pin assignment needs to be done if the designer already knows the device to be used. However, when creating a VIRTUAL design, only the variable names that will later be assigned to pins need to be specified. The area that normally contains the pin numbers will be left blank.

2.1.2. Defining Intermediate Variables

Intermediate variables are variables that are assigned an equation but are not assigned to a PIN or NODE. These are used to define equations that are utilized by many variables or to provide a clearer understanding of the design.

2.1.3. Using Indexed Variables

Variable names that end in a decimal number from 0 to 31 are referred to as indexed variables. They can be used to represent a group of address lines, data lines or other sequentially numbered items. When indexed variables are used in bit-field operations, the variable with index number 0 is always the lowest-order bit.

2.1.4. Using Number Bases

All operations involving numbers in the CUPL compiler are performed with 32-bit accuracy. Therefore, numbers may have values from 0 to 2^{32}-1. A number may be represented in any of the four common bases: binary, octal, decimal or hexadecimal. The default base for all numbers used in the source file is hexadecimal, except for device pin numbers and indexed variables, which are always decimal. Binary, octal and hexadecimal numbers can have "don't care" (“X”) values intermixed with numerical values.

Table 2-1. Using Number Bases

Number	Base	Decimal Value
‘b’0	Binary	0
‘B’1101	Binary	13
‘O’663	Octal	435
‘D’92	Decimal	92
‘h’BA	Hexadecimal	186
‘O’[300..477]	Octal (range)	192..314
‘H’7FXX	Hexadecimal (range)	32512..32767

© 2026 Microchip Technology Inc. and its subsidiaries

2.1.5. Using List Notation

A list is a shorthand method for defining groups of variables. It is commonly used in pin and node declarations, bit field declarations, logic equations and set operations. Square brackets are used to delimit items in the list.

2.1.6. Using Bit Fields

A bit field declaration assigns a single variable name to a group of bits. After making a bit field assignment using the *FIELD* keyword, the name can be used in an expression; the operation specified in the expression is applied to each bit in the group. When a FIELD statement is used, the compiler generates a single 32-bit field internally. This field is used to represent the variables in the bit field, with each bit representing one member of the bit field. The bit number that represents a member of a bit field is the same as the index number when indexed variables are used. This means that A0 will always occupy bit 0 in the bit field. Bit fields are mainly used for defining and manipulating address and data buses.

Figure 2-1. Using the FIELD Statement

```
FIELD ADDRESS = [A7, A6, A5, A4, A3, A2, A1, A0];
FIELD DATA = [D7..D0];
FIELD Mode = [Up, Down, Hold];
```

2.2. Usage of the Language Syntax

This section discusses the logic and arithmetic operators and functions needed to create a Boolean equation design.

2.2.1. Using Logical Operators

Four standard logical operators are available for use: NOT, AND, OR and XOR. The following table lists the operators and their order of precedence, from highest to lowest.

Table 2-2. Logical Operators

Operator	Examples	Description	Precedence
!	!A	NOT	1
&	A & B	AND	2
#	A # B	OR	3
$	A $ B	XOR	4

2.2.2. Using Arithmetic Operators and Functions

Six standard arithmetic operators are available for use in $repeat and $macro commands. The following table lists these operators and their order of precedence, from highest to lowest.

Table 2-3. Arithmetic Operators

Operator	Examples	Description	Precedence
**	2**3	Exponentiation	1
*	2*I	Multiplication	2
/	4/2	Division	2
%	9%8	Modulus	2
+	2+4	Addition	3
-	4-I	Subtraction	3

One arithmetic function is available for use in arithmetic expressions used in $repeat and $macro commands. The following table shows the arithmetic function and its bases.

© 2026 Microchip Technology Inc. and its subsidiaries

Table 2-4. Arithmetic Function and Bases

Function	Base
LOG2	Binary
LOG8	Octal
LOG16	Hexadecimal
LOG	Decimal

The LOG function returns an integer value. For example:

```
LOG2(32)= 5 <==> 2**5 = 32
LOG2(33)= ceil(5.0444) = 6 <==> 2**6 = 64
```

Ceil(x) returns the smallest integer that is not less than x.

2.2.3. Using Variable Extensions

Extensions can be added to variable names to indicate specific functions associated with major nodes within a programmable device, including capabilities such as flip-flop description and programmable tri-state enables. The compiler checks the usage of the extension to determine whether it is valid for the specified device and whether its usage conflicts with any other extension used. CUPL uses these extensions to configure the macrocells within a device. This way, the designer does not have to know which fuses control which in the macrocells. A complete list of CUPL extensions can be found in the *CUPL Programmer's Reference Guide* in the Extensions section of the CUPL Language Reference chapter.

Table 2-5. Variable Extensions for Microchip SPLDs/CPLDs

Extension	Side Used	Description
.AP	L	Asynchronous preset of flip-flop
.AR	L	Asynchronous reset of flip-flop
.CE	L	CE input of enabled D-CE type flip-flop
.CK	L	Programmable clock of flip-flop
.CKMUX	L	Clock multiplexer selection
.D	L	D input of D-type flip-flop
.DFB	R	D registered feedback path selection
.DQ	R	Q output of D-type flip-flop
.INT	R	Internal feedback path for registered macrocell
.IO	R	Pin feedback path selection
.J	L	J input of JK-type output flip-flop
.K	L	K input of JK-type output flip-flop
.L	L	D input of transparent latch
.LE	L	Programmable latch enable
.LQ	R	Q output of transparent input latch
.OE	L	Programmable output enable
.R	L	R input of SR-type output flip-flop
.S	L	S input of SR-type output flip-flop
.SP	L	Synchronous preset of flip-flop
.T	L	T input of toggle output flip-flop
.TFB	R	T registered feedback path selection

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 2-2. Circuit Illustrating Extension

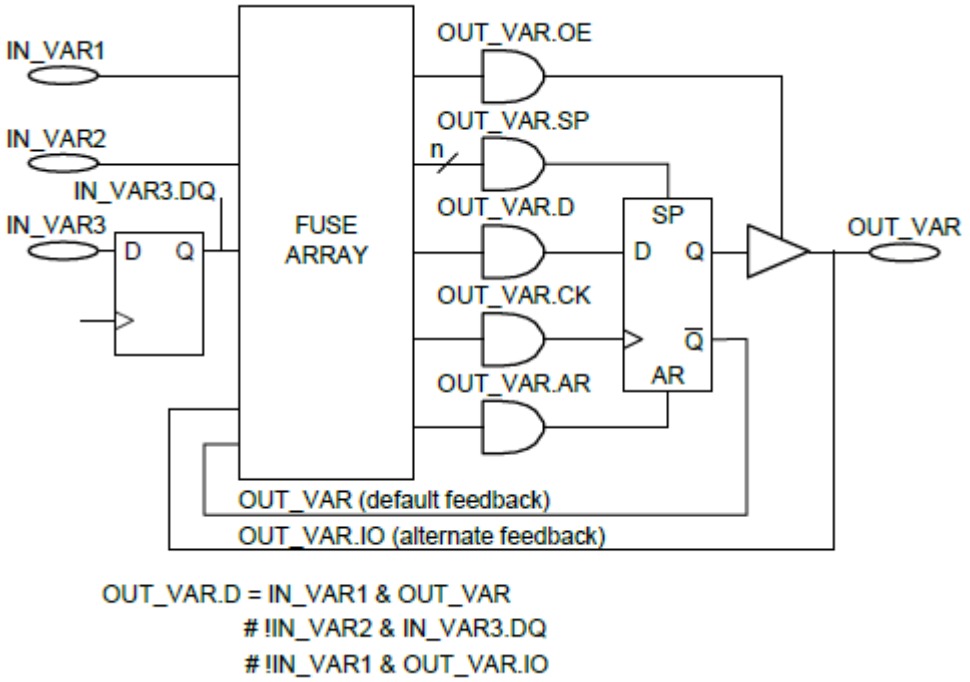
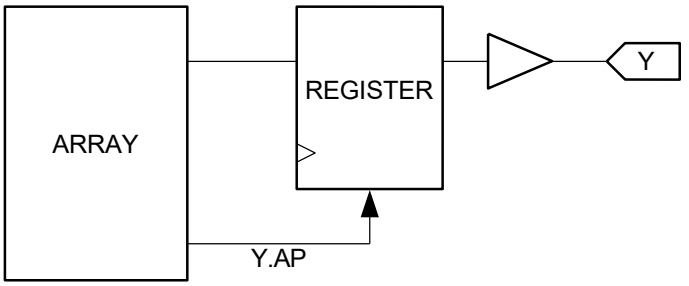


Figure 2-2 shows the use of extensions. Note that this figure does not represent an actual circuit but demonstrates how to use extensions to write equations for different functions in a circuit.

.AP Extension

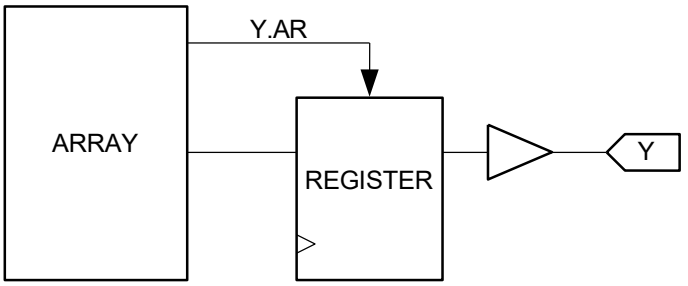
The .AP extension is used to set the Asynchronous Preset of a register to an expression. For example, the equation "Y.AP = A&B;" causes the register to be asynchronously preset when A and B are logically true. This feature is supported by the ATF15XX family of devices.



.AR Extension

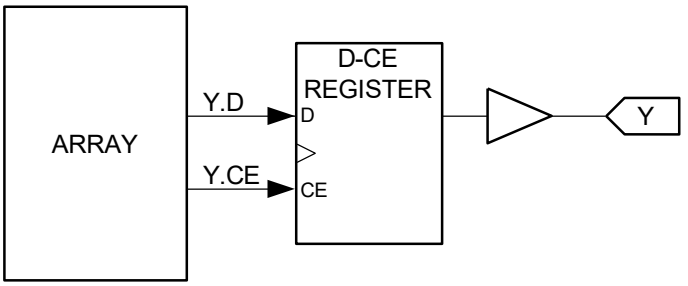
The .AR extension is used to define the expression for Asynchronous Reset for a register. This is used in devices that have one or more product terms connected to the Asynchronous Reset of the register. Devices that have pin-controlled reset inputs, such as the Microchip ATF15XX family, also use this suffix.

© 2026 Microchip Technology Inc. and its subsidiaries



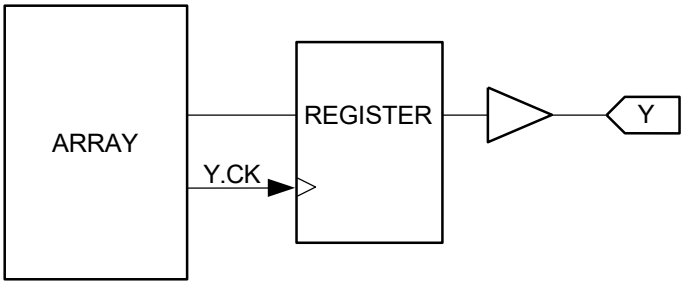
.CE Extension

The .CE extension is used for D-type registers that have a clock enable input (D-CE registers). It serves to specify the input to the clock enable term of the register. In devices that have D-CE registers, such as the ATF2500C and the ATF15XX, the CE terms, if not used, must be set to binary 1 so that the registers behave as D registers. Normally, the CUPL compiler or ATF15XX fitter will automatically do this for you.



.CK Extension

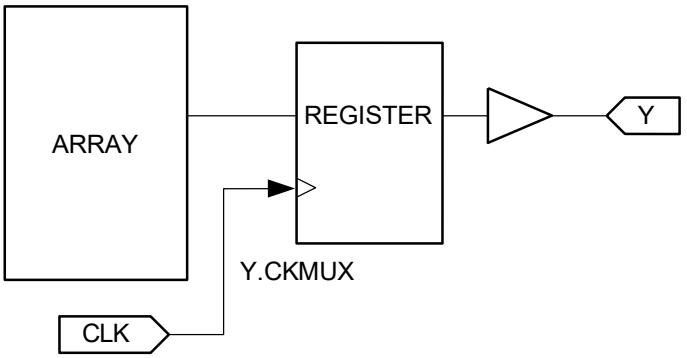
The .CK extension is used to select a product term driven clock. Some devices have the capability to connect the clock for a register to one or more pins or to a product term. The .CK extension will select the product term. Use this suffix to connect the clock for a register to the dedicated clock pin for any Microchip PLD that has this feature, except the ATF750C (refer to the .CKMUX Extension).



.CKMUX Extension

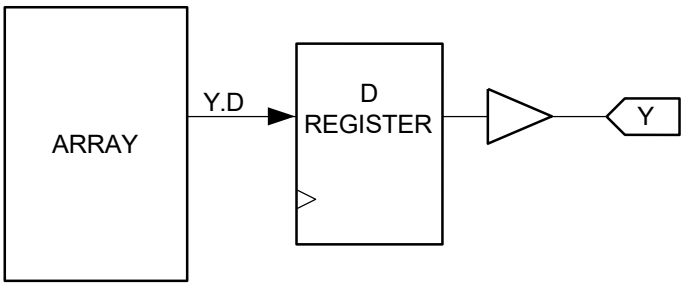
The .CKMUX extension is used to connect the pin clock to the register. This extension is used on the ATF750C device. Using this extension can reduce the number of products needed to implement the design into the Microchip PLD and can increase the AC timing performance of the design. Other Microchip PLDs, such as the ATF2500C and ATF15XX family, also have dedicated clock pins, but they are not specified with the .CKMUX extension. They use the .CK extension instead.

© 2026 Microchip Technology Inc. and its subsidiaries



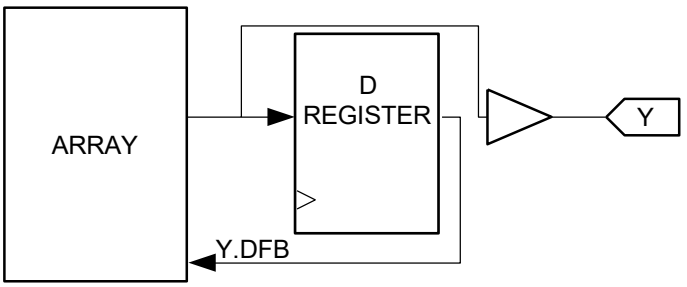
.D Extension

The .D extension is used to specify the D input to a D-type register. This option causes the compiler to configure the macrocells in the Microchip PLD device to D-type registers. For Microchip PLDs such as the ATF16V8B/C, ATF22V10B/C, ATF750C, and ATF2500C, the .D extension must be used for registered logic. Otherwise, CUPL will generate an error.



.DFB Extension

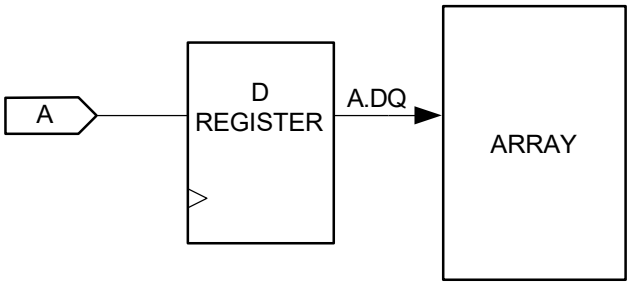
The .DFB extension is used when the macrocell on a Microchip device is configured for a combinatorial output, but the D register still remains connected to the output. This configuration is supported on the ATF750C and ATF2500C devices. The .DFB allows the registered representation of the combinatorial output to be fed back internally into the Microchip PLD.



.DQ Extension

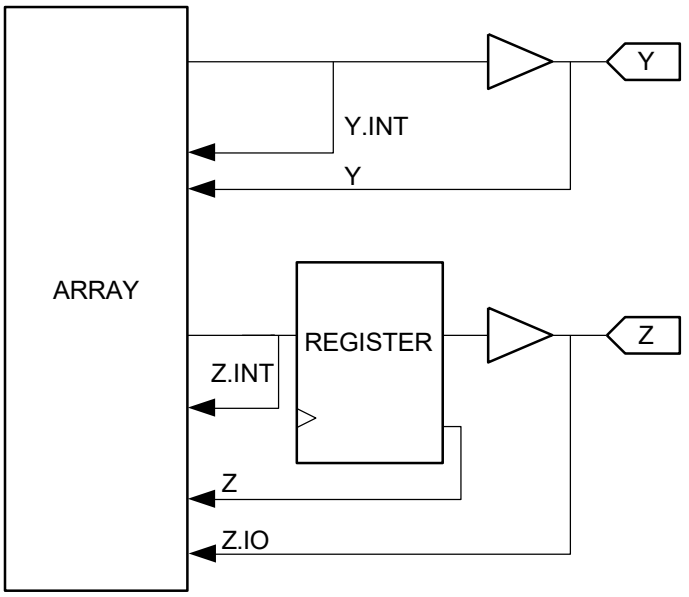
The .DQ extension is used to specify an input D register. The use of the .DQ extension actually configures the input as registered. The .DQ extension is not used to specify the Q output from an output D register. This feature is available on the ATF15XX CPLDs.

© 2026 Microchip Technology Inc. and its subsidiaries



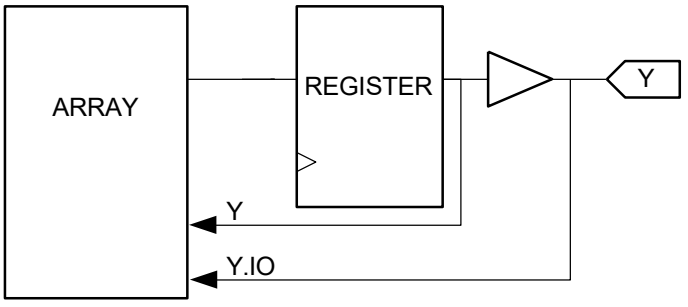
.INT Extension

The .INT extension is used for selecting an internal feedback path. This can be used to specify the buried combinatorial feedback path for either a registered or combinatorial output. This feature is available for the ATF15XX CPLDs.



.IO Extension

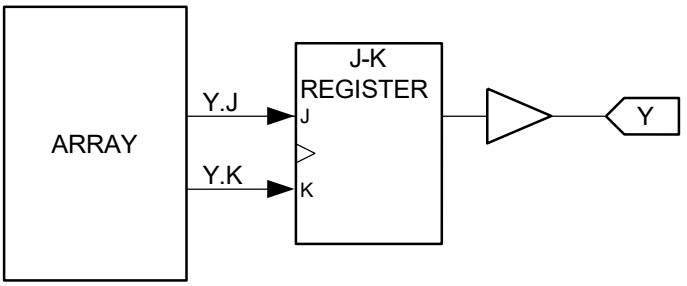
The .IO extension is used to select pin feedback within the macrocell. This is useful when a design requires using an I/O pin as an input and also requires buried logic to be used within the same macrocell. It is also useful for implementing bi-directional outputs in CUPL. For examples of how to implement bi-directional I/O in CUPL, refer to the *Tips on Using Test Vectors* application note.



© 2026 Microchip Technology Inc. and its subsidiaries

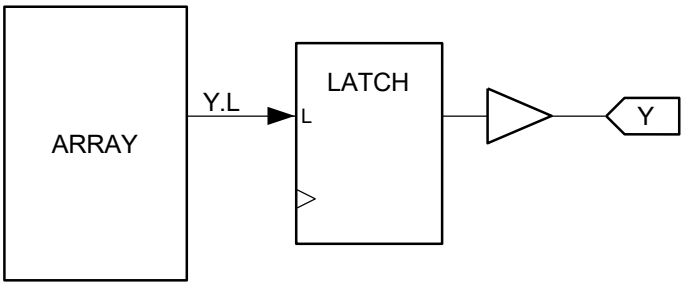
.J and .K Extensions

The .J and .K extensions are used to specify J and K inputs to a J-K register. Logic equations using these extensions can be written with the REGISTER SELECT keyword when the D or T-type target register is specified.



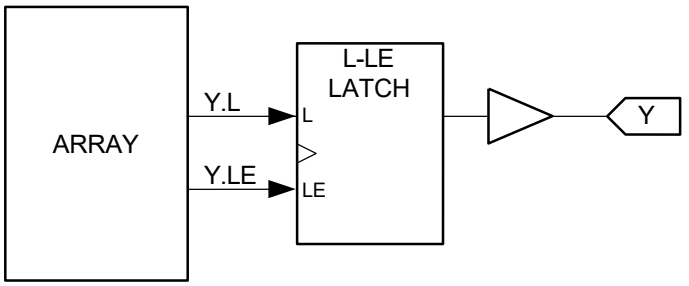
.L Extension

The .L extension is used to specify input into a latch. This extension is required to use the level-triggered latch feature available on the ATF15XX family of devices. The use of the .L extension causes the compiler to configure the macrocell as a latched output.



.LE Extension

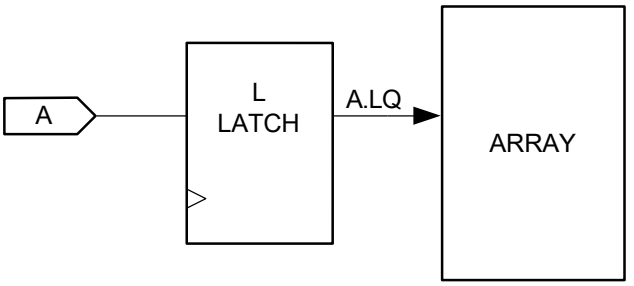
The .LE extension is used to specify the latch enable equation for a latch. It is required for designs using the level-triggered latch feature available on the ATF15XX family of devices. The .LE extension causes a product term to be connected to the latch enable.



.LQ Extension

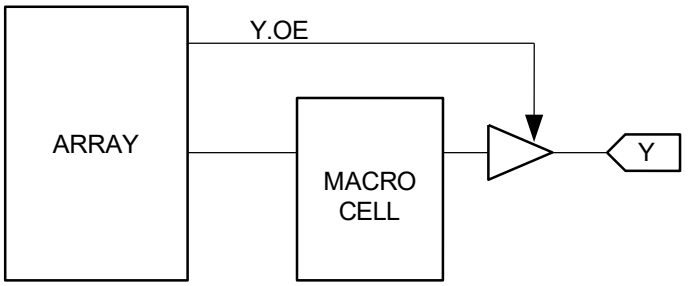
The .LQ extension is used to specify an input latch. The use of the .LQ extension configures the input as latched. The .LQ extension is not used to specify the Q output from an output latch. This feature is available on the ATF15XX family of devices.

© 2026 Microchip Technology Inc. and its subsidiaries



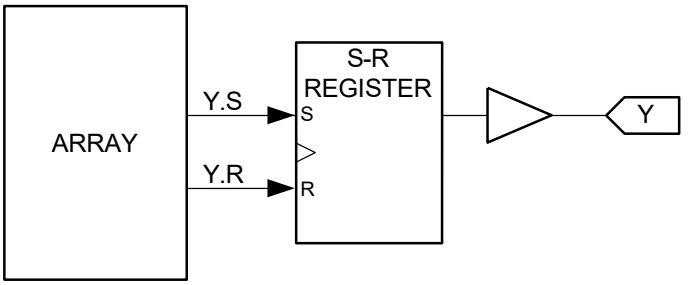
.OE Extension

The .OE extension is used to specify a product term-driven output enable signal. It is required for using bi-directional I/O and the individually programmable output-enable product terms available on the ATF750C, ATF2500C and ATF15XX family of devices. Microchip PLDs that have pin-controlled OE inputs, such as the ATF15xx family, also use this suffix.



.S and .R Extensions

The .R and .S extensions are used to specify R (Reset) and S (Set) inputs to an SR register. Logic equations for the ATF15XX family can be written using these extensions if the REGISTER SELECT keyword is used and a D or T-type target register is specified.



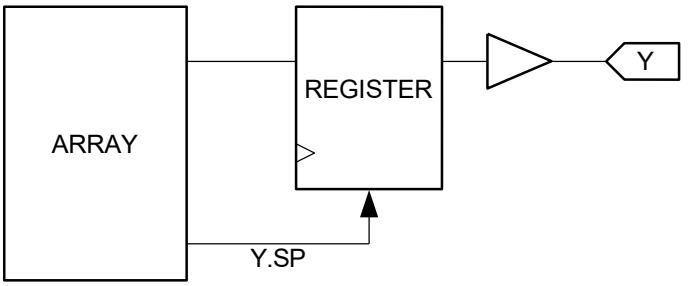
.SP Extension

The .SP extension is used to set the Synchronous Preset of a register to an expression. For example, the equation:

Y.SP = A & B ; /* **A** and **B** are inputs */

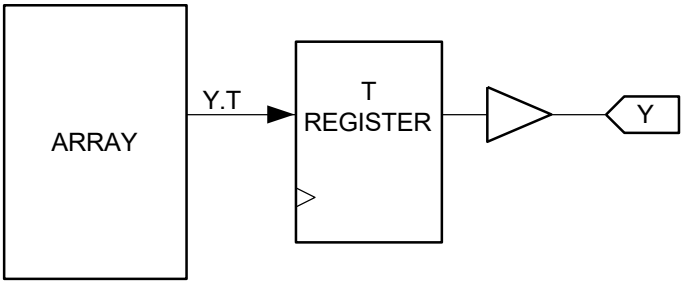
causes the output **Y** to be preset synchronously with the local clock used in the macro-cell when inputs **A** and **B** are true. This feature is supported on the ATF22V10B/C, ATF750C and ATF2500C devices, which share synchronous preset product terms.

© 2026 Microchip Technology Inc. and its subsidiaries



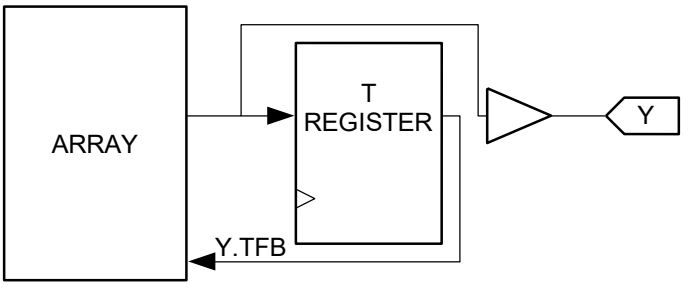
.T Extension

The .T Extension specifies the T input for a T register. The use of the T extension causes the compiler to configure the macrocell as a T-type register. This feature is supported on the ATF750C/2500C and ATF15XX family of devices.



.TFB Extension

The .TFB extension is used when the macrocell on a Microchip device is configured for a combinatorial output, but the T register still remains connected to the output. This configuration is supported on the ATF750C and ATF2500C devices. The .TFB allows the registered representation of the combinatorial output to be fed back internally into the Microchip PLD.



© 2026 Microchip Technology Inc. and its subsidiaries

2.2.4. Defining Logic Equations

Logic equations are the building blocks of the CUPL language. The form for logic equations is as follows:

```
[!] var [.ext] = exp;
```

where:

var is a single variable or a list of indexed or non-indexed variables defined according to the rules for list notation. When a variable list is used, the expression is assigned to each variable in the list.

.ext is an optional variable extension to assign a function to the major nodes inside programmable devices.

exp is an expression; that is, a combination of variables and operators.

= is the assignment operator; it assigns the value of an expression to a variable or set of variables.

! is the complement operator.

In standard logic equations, normally only one expression is assigned to a variable. The *APPEND* statement enables multiple expressions to be assigned to a single variable. The APPENDed logic equation is logically ORed with the original equation for that variable. The format for using the APPEND statement is identical to defining a logic equation, except the keyword APPEND appears before the logic equation begins.

Place logic equations in the “Logic Equation” section of the source file provided by the template file.

2.2.5. Using Set Operations

All operations that are performed on a single bit of information, for example, an input pin, a register, or an output pin, may be applied to multiple bits of information grouped into sets. Set operations can be performed between a set and a variable or expression, or between two sets.

The result of an operation between a set and a single variable (or expression) is a new set in which the operation is performed between each element of the set and the variable (or expression).

When an operation is performed on two sets, the sets must be the same size (that is, contain the same number of elements). The result of an operation between two sets is a new set in which the operation is performed between elements of each set.

When numbers are used in set operations, they are treated as sets of binary digits. A single octal number represents a set of three binary digits, and a single decimal or hexadecimal number represents a set of four binary digits.

2.2.6. Using Equality Operations

Unlike other set operations, the equality operation evaluates to a single Boolean expression. It checks for bit equality between a set of variables and a constant. The bit positions of the constant number are checked against the corresponding positions in the set. Where the bit position is a binary 1, the set element is unchanged. Where the bit position is a binary 0, the set element is negated. Where the bit position is a binary X, the set element is removed. The resulting elements are then ANDed together to create a single expression.

The equality operator can also be used with a set of variables that are to be operated upon identically. For example, the following three expressions:

```
[A3, A2, A1, A0]:&
[B3..B0]:#
[C3, C2, C1, C0]:$
```

are equivalent, respectively, to:

© 2026 Microchip Technology Inc. and its subsidiaries

```
A3 & A2 & A1 & A0
B3 # B2 # B1 # B0
C3 $ C2 $ C1 $ C0
```

2.2.7. Using Range Operations

The range operation is similar to the equality operation, except that the constant field is a range of values instead of a single value. The check for bit equality is made for each constant value in the range.

First, define the address bus as follows:

```
FIELD address = [A3..A0];
```

Then write the *RANGE* equation:

```
select = address:[C..F];
```

This is equivalent to the following equation:

```
select = address:C # address:D # address:E # address:F;
```

2.3. Advanced Language Syntax

This section describes the advanced CUPL language syntax. It explains how to use truth tables, state machines, conditional statements and user-defined functions to create a PLD design.

2.3.1. Defining Truth Tables

Sometimes the clearest way to express logic descriptions is in tables of information. CUPL provides the *TABLE* keyword to create tables of information. First, define relevant input and output variable lists, and then specify one-to-one assignments between decoded values of the input and output variable lists. “Don’t care” values are supported for the input decode value, but not for the output decoded value.

A list of input values can be specified to make multiple assignments in a single statement. The following block describes a simple hex-to-BCD code converter:

```
FIELD input = [in3..0];
FIELD output = [out3..0];
TABLE input => output {
                  0=> 00; 1=>01; 2=>02; 3=>03;
                  4=>04; 5=>05; 6=>06; 7=>07;
                  8=>08; 9=>09; A=>10; B=>11;
                  C=>12; D=>13; E=>14; F=>15;
}
```

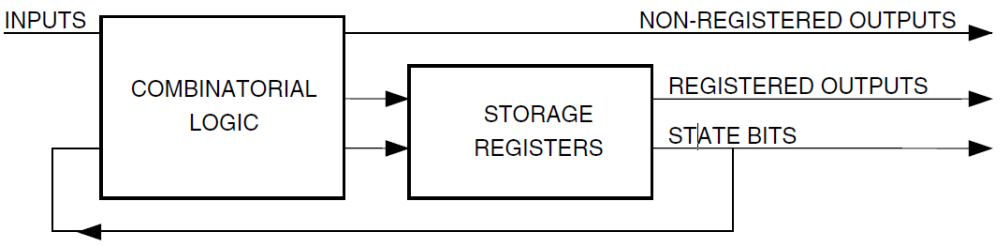
2.3.2. Defining State Machines

A state machine is a digital device that traverses a predetermined sequence of states in an orderly fashion. A synchronous state machine is a logic circuit with flip-flops. Because its output can be fed back to its own or to another flip-flop’s input, a flip-flop’s input value may depend on both its own output and the output of other flip-flops; consequently, its final output value depends on its own previous values, as well as those of other flip-flops.

The CUPL state-machine model, as shown in Figure 2-3, uses six components: inputs, combinatorial logic, storage registers, state bits, registered outputs and non-registered outputs.

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 2-3. State Machine Model



Inputs - are signals entering the device that originate from some other device.

Combinatorial Logic is any combination of logic gates (usually AND-OR) that produces an output signal that is valid Tpd (propagation delay time) nanoseconds after any of the signals driving these gates change. Tpd is the delay between the initiation of an input or feedback event and the occurrence of a non-registered output.

State Bits are storage register outputs that are fed back to drive the combinatorial logic. They contain the present-state information.

Storage Registers are any flip-flop elements that receive their inputs from the state machine's combinatorial logic. Some registers are used for state bits, while others are used for registered outputs. The registered output is valid Tco (clock to out time) nanoseconds after the clock pulse occurs. Tco is the time delay between the initiation of a clock signal and the occurrence of a valid flip-flop output.

To implement a state machine, CUPL supplies a syntax that allows the description of any function in the state machine. The *SEQUENCE* keyword identifies the outputs of a state machine and is followed by statements that define the function of the state machine. The SEQUENCE keyword causes the storage registers and registered output types generated to be the default type for the target device. Along with the SEQUENCE keyword are the *SEQUENCED, SEQUENCEJK, SEQUENCERS* and *SEQUENCET* keywords. Respectively, they force the state registers and registered outputs to be generated as D, J-K, S-R, and T-type flip-flops. The format for the SEQUENCE syntax is as follows:

```
SEQUENCE state_var_list {
          PRESENT state_n0
                    IF(condition1)NEXT state_n1;
                    IF (condition2) NEXT state_n2 OUT out_n0;
                    DEFAULT NEXT state_n0;
          PRESENT state_n1
                    NEXT state_n2;
          .
          .
          .
          PRESENT state_nn statements;
}
```

where

state_var_list is a list of the state bit variables used in the state machine block. The variable list can be represented by a field variable.

state_n is the state number and is a decode value of the **state_variable_list** and must be unique for each present statement.

statements are any of the conditional, next, or output statements described in the following subsection.

© 2026 Microchip Technology Inc. and its subsidiaries

2.3.3. Defining Multiple State Machines

The CUPL syntax allows for more than one state machine to be defined within the same PLD design. When multiple state machines are defined, occasionally the designer would like to have the state machines communicate with each other. That is, when one state machine reaches a certain state, another state machine may begin. There are two methods of accomplishing state machine communication: using set operations on the state bits or defining a “global” register that can be accessed by both state machines.

In one state machine, a conditional statement can contain another state machine’s name followed by a state number or range of state numbers. The conditional statement will become TRUE when the other state machine reaches that particular state or states. The same case is true when using a register that is accessed by multiple state machines. However, this method requires the use of one of the device's output or buried registers. Depending on the situation, the global register could also be combinatorial, which may make a difference as to when the state machine receives the information from another state machine.

2.3.4. Using Condition Statement

The *CONDITION* syntax provides a higher-level approach to specifying logic functions than writing standard Boolean logic equations for combinatorial logic. The format is as follows:

```
CONDITION          {
 IF                expr0   OUT     var;
 -
 -
 IF                exprn   OUT     var;
 DEFAULT                   OUT     var;
 }
```

The CONDITION syntax is equivalent to the asynchronous conditional output statements of the state machine syntax, except that there is no reference to any particular state. The variable is logically asserted whenever the expression or DEFAULT condition is met.

2.3.5. Defining a Function

The *FUNCTION* keyword permits the creation of personal keywords by encapsulating some logic as a function and giving it a name. This name can then be used in a logic equation to represent the function. The format for user-defined functions is as follows:

```
FUNCTION  name  ([Parameter0,...,Parametern])
{     body     }
```

The statements in the body may assign an expression to the function or may be unrelated equations.

When using optional parameters, the number of parameters in the function definition and in the reference must be identical. The parameters defined in the body of the function are substituted for the parameters referenced in the logic equation. The function invocation variable is assigned an expression according to the body of the function. If no assignment is made in the body statements, the function invocation variable is assigned the value of ‘h’0.

2.3.6. MIN Declaration Statements

The **MIN** declaration permits specifying different levels for different outputs in the same design, such as no reduction for outputs requiring redundancy or containing product terms (to avoid asynchronous hazard conditions), and maximum reduction for a state machine application.

The **MIN** declaration statement overrides, for specified variables, the minimization level specified on the command line when running CUPL. The format is as follows:

© 2026 Microchip Technology Inc. and its subsidiaries

```
MIN var [.ext] = level ;
```

MIN is a keyword to override the command line minimization level.

var is a single variable declared in the file or a list of variables grouped using the list notation; that is,

```
MIN [var, var, ... var] = level
```

.ext is an optional extension that identifies the function of the variable
level is an integer between 0 and 4.

; is a semicolon to mark the end of the statement.

Levels 0 to 4 correspond to the minimization levels available: None, Quick, Quine McClusky, Presto, Espresso.

The following are examples of valid MIN declarations:

```
MIN async_out = 0; /* no reduction */
MIN [outa, outb] = 1; /* Quine McClusky reduction */
MIN count.d = 4; /* Espresso reduction */
```

Note that the last declaration in the example above uses the .D extension to specify that the registered output variable is the one to be reduced.

© 2026 Microchip Technology Inc. and its subsidiaries

3. Using WinCUPL II

WinCUPL II, a *Universal Compiler for Programmable Logic,* is a versatile and powerful logic compiler that can be used to create very sophisticated logic designs for Microchip SPLDs and CPLDs. It supports design entry, compilation and functional simulation for CUPL designs.

3.1. Overview

The WinCUPL II software includes the following tools:

WinCUPL II IDE - A powerful front-end user interface for all of the WinCUPL II tools, including the compiler and simulator.

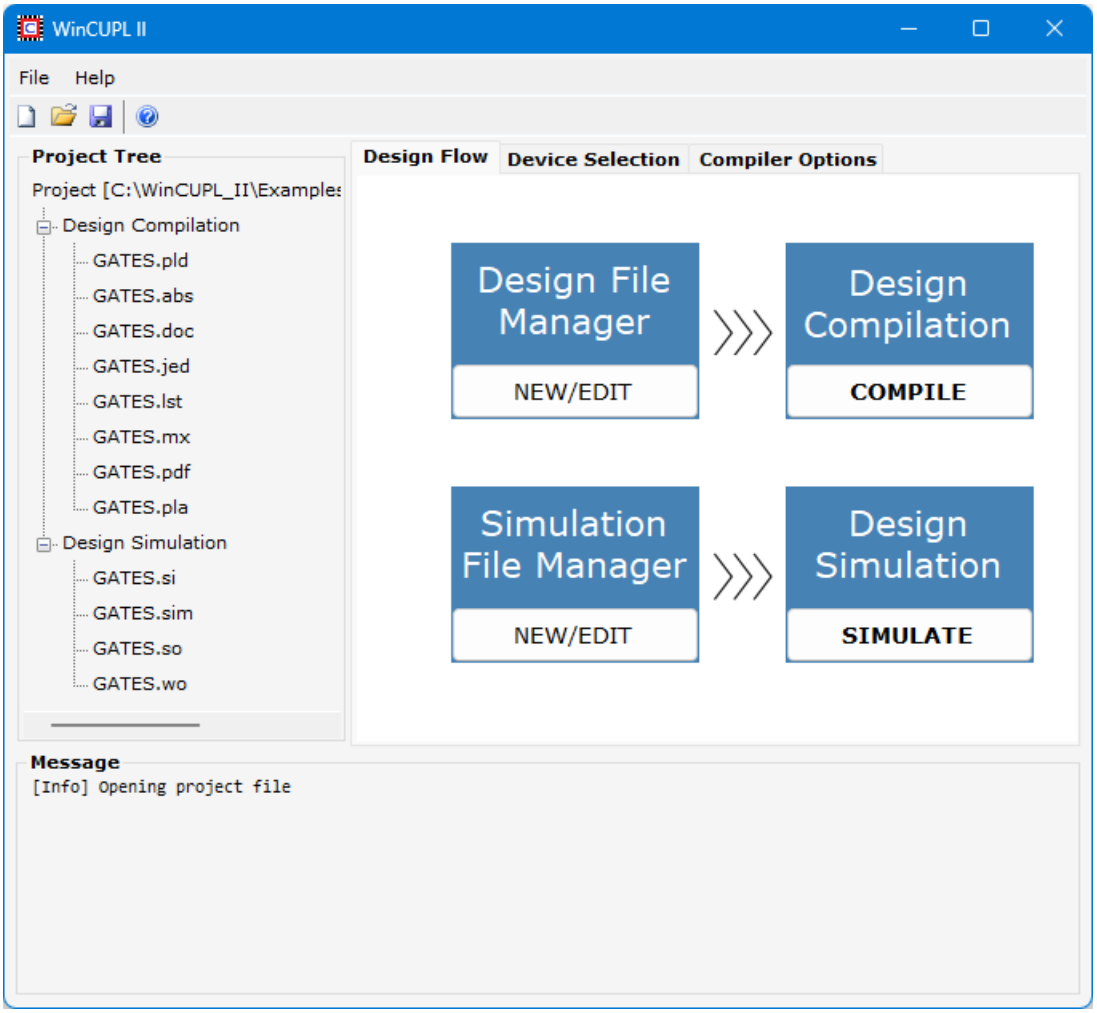
Text Editor - Built-in text editor to create CUPL design and simulation source files with syntax highlighting.

CUPL Compiler - Logic descriptions written in the CUPL language are compiled and can be assigned to specific logic devices (PLDs). Upon compilation, the CUPL compiler searches its libraries and creates a file that can be downloaded to a device programmer. From this point, the PLD can be programmed.

Simulator - Designs can be functionally simulated before they are put into production. The CUPL simulator (CSIM) compares the expected values to the actual values calculated during CUPL operation.

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 3-1. WinCUPL II's Main Screen



3.2. Data and Design Flows

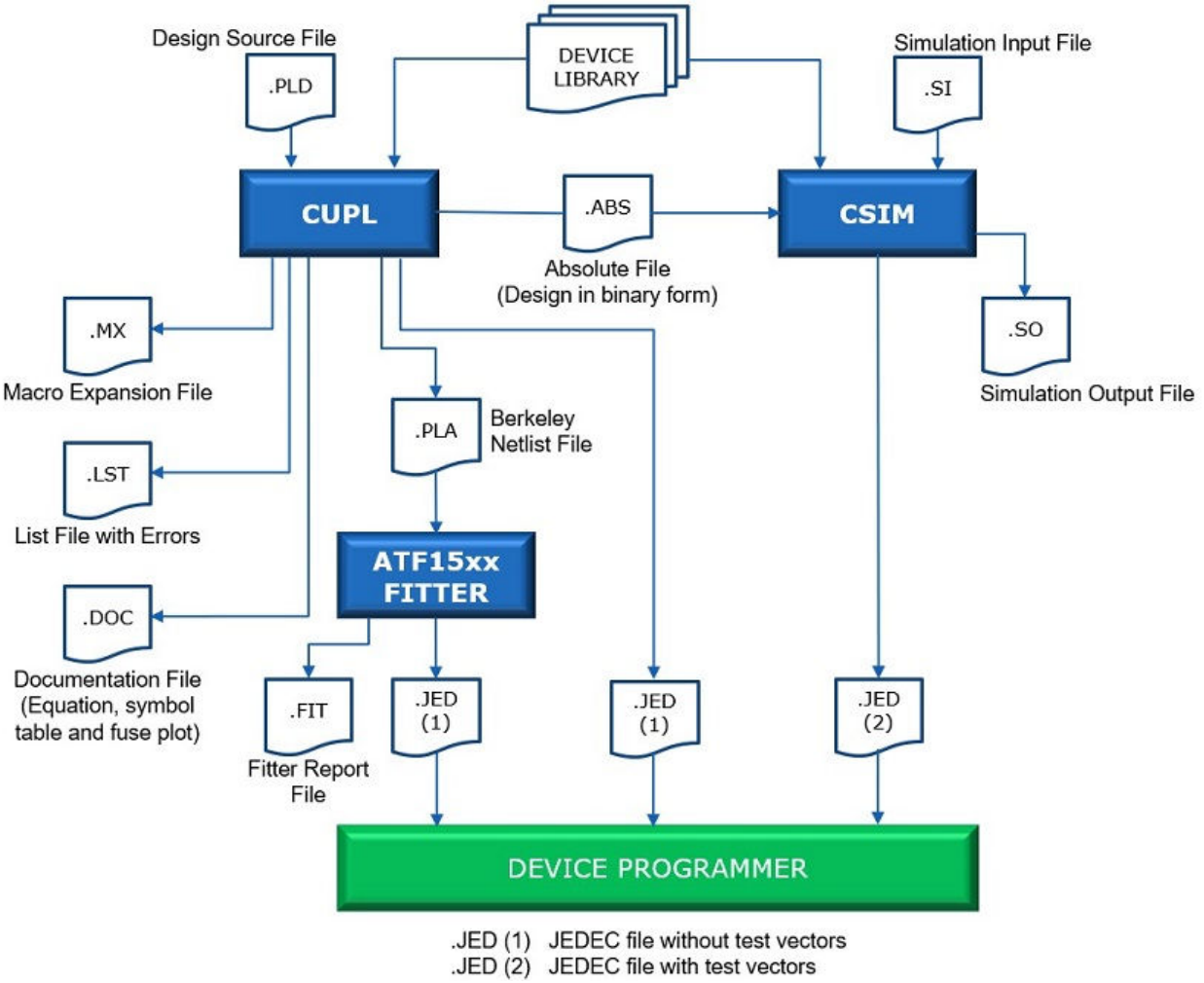
Figure 3-2 illustrates the data flow for creating a design and implementing the design using CUPL.

First, a logic description is created using the CUPL language manually with the WinCUPL II source editor.

Then, the design is compiled to create a fusemap file for downloading to a device programmer. Optionally, a test specification file may be created to verify the design. The CSIM functional simulator is executed to compare the expected values in the test file to the actual values in the absolute file created by CUPL. When the simulation is completed without any errors, the verified test vectors can be appended to the download JEDEC file generated by CUPL.

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 3-2. WinCUPL II Data Flow



The CUPL design flow via WinCUPL II is described below:

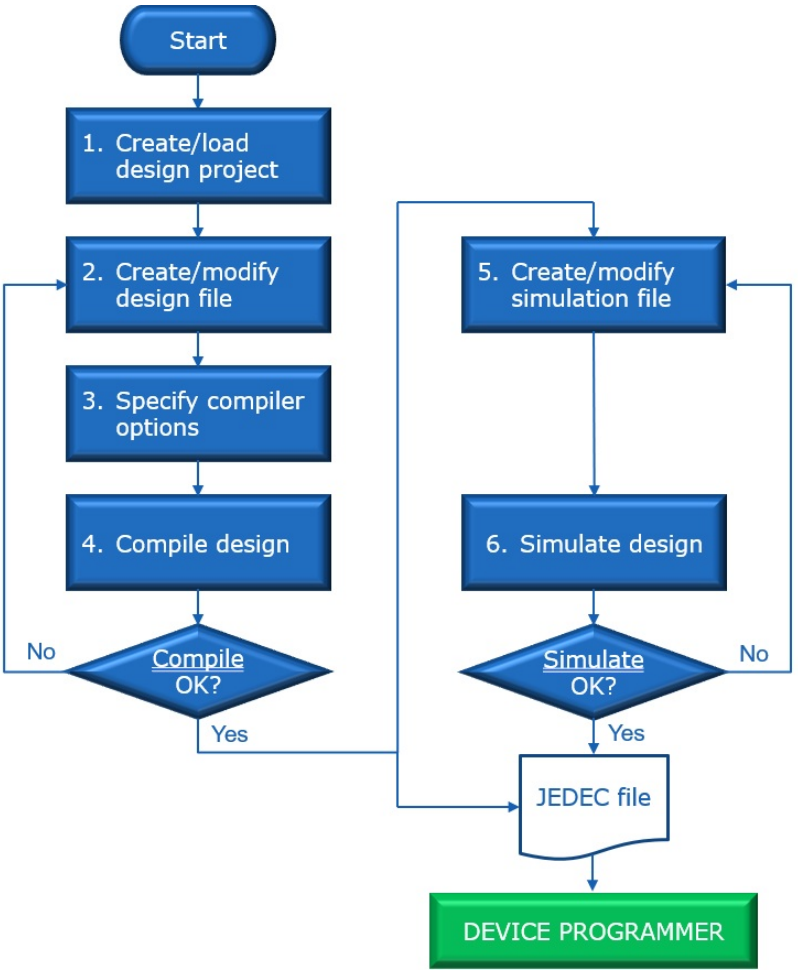
1. **Create or load project file** - To create a new project, click on the New Project shortcut button in the main WinCUPL II window to launch the New Project Wizard. In the New Project Wizard, select the target device type, device package type and device option. A new project file with the extension .WCP will be created, and it will contain all the project related information. To load an existing project, click on the Open Project shortcut button in the main WinCUPL II window and then select the project file (.WCP) that you would like to load into WinCUPL II. After creating or loading a design project into WinCUPL II, the target device type, package type or device option can be changed under the Device Selection tab.
2. **Create or modify design file** - Click on the New/Edit button under Design File Manager to open a new or existing CUPL design file (.PLD) in the WinCUPL II text editor. After entering the logic equations into the CUPL design file, save the file and close the text editor to return to the main WinCUPL II window.
3. **Specify compiler options** - Select the desired compiler options under the Compiler Options tab.
4. **Compile design** - Click on the COMPILE button under Design Compilation to compile the CUPL design. The messages generated by the compiler will be displayed in the Message pane of the main WinCUPL II window, as well as in the LOG file.
5. **Create or modify simulation file** - Click on the New/Edit button under Simulation File Manager to open a new or existing CUPL simulation input file (.SI) in the WinCUPL II text editor. After

© 2026 Microchip Technology Inc. and its subsidiaries

entering the simulation inputs into the simulation input file, save the file and close the text editor to return to the main window.

6. **Simulate design** - Click on the SIMULATE button under Design Simulation to simulate the CUPL design. The outputs of the simulation will be contained in the simulation output file (.SO).

Figure 3-3. WinCUPL II Design Flow



© 2026 Microchip Technology Inc. and its subsidiaries

4. WinCUPL II Tutorial

This section introduces the features and usage of the WinCUPL II development software for the Microchip Simple and Complex Programmable Logic Devices (SPLDs/CPLDs). It describes the entire development process, from creating a new project in WinCUPL II and compiling a CUPL design to running a functional simulation on a CUPL design.

4.1. 2-Input AND Gate

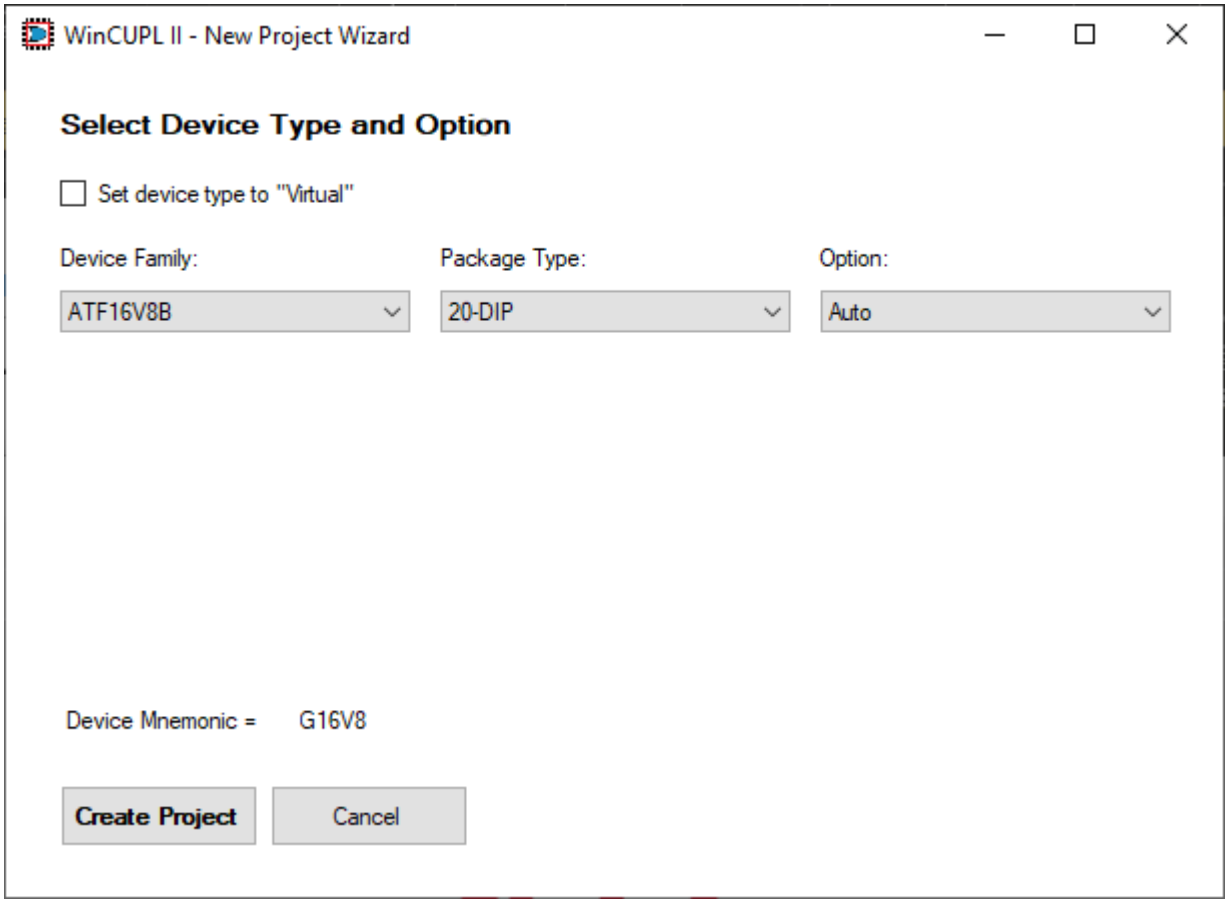
Launch WinCUPL II by double clicking on the WinCUPL II icon.

4.1.1. Create New Project

In WinCUPL II, select **File ➔ New Project** or click on the **New Project** shortcut button to open the **New Project Wizard** to create a new project.

In the **New Project Wizard**, select **ATF16V8B** in the **Device Family** field, **20-DIP** in the **Package Type** field and **Auto** in the **Option** field, as shown below.

Figure 4-1. WinCUPL II New Project Wizard



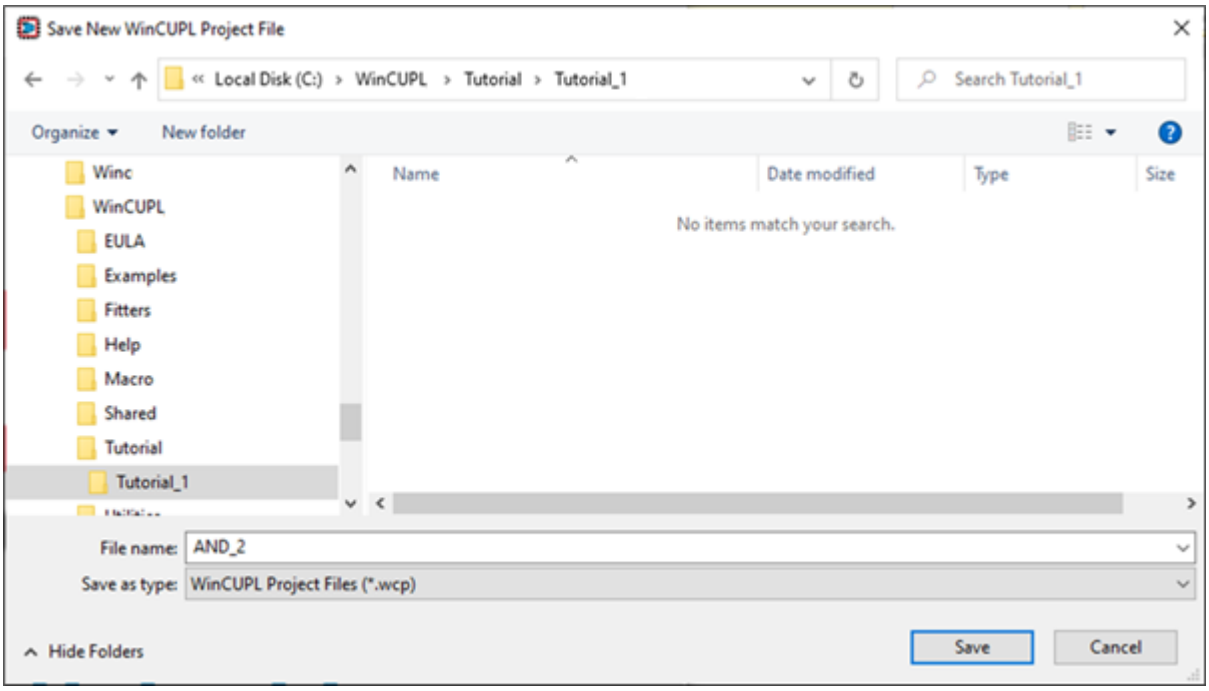
In the **New Project Wizard**, select the **Create Project** button to launch the **Save New WinCUPL Project File** browser window. In this window, click the **New Folder** button and create a new folder named **Tutorial** under the WinCUPL II installation directory (for example, C:\WinCUPL\Tutorial). Click the **New Folder** button again and create a new folder named **Tutorial_1**. Open the newly created **Tutorial_1** folder, enter **AND_2** in the **File name** field, and click the **Save** button to create a new project file (**AND_2.wcp**) for this design.

© 2026 Microchip Technology Inc. and its subsidiaries

Note:

Path and file names for WinCUPL II projects cannot exceed 127 characters or contain spaces.

Figure 4-2. File Browser to Save New Project File

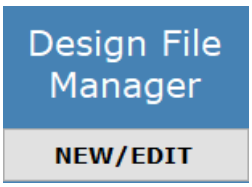


4.1.2. Create New CUPL Design File

The next step in the design development process is to create a new CUPL design (.PLD) file using the built-in text editor in WinCUPL II.

In WinCUPL II, select the **Design Flow** tab and then click the **NEW/EDIT** button under **Design File Manager** to launch the text editor. The software will automatically create a new CUPL design file named “AND_2.pld”, based on the file name used for the project file.

Figure 4-3. Design File Manager



In the text editor, select **File → Save** to save the CUPL design file. This design file will be saved as **AND_2.pld** in the AND_2 project directory (for example, ~\WinCUPL\Tutorial\Tutorial_1).

© 2026 Microchip Technology Inc. and its subsidiaries

Enter the following CUPL code for the AND_2 design into the text editor:

```
Name          AND_2;
PartNo        ;
Date          ;
Revision      ;
Designer      ;
Company       ;
Assembly      ;
Location      ;
Device        ;

/* INPUT PINS */
PIN  2 = A;          /* 1st AND gate input */
PIN  3 = B;          /* 2nd AND gate input */

/* OUTPUT PINS */
PIN 19 = Q;          /* AND gate output */

/* LOGIC EQUATION */
Q = A & B;
```

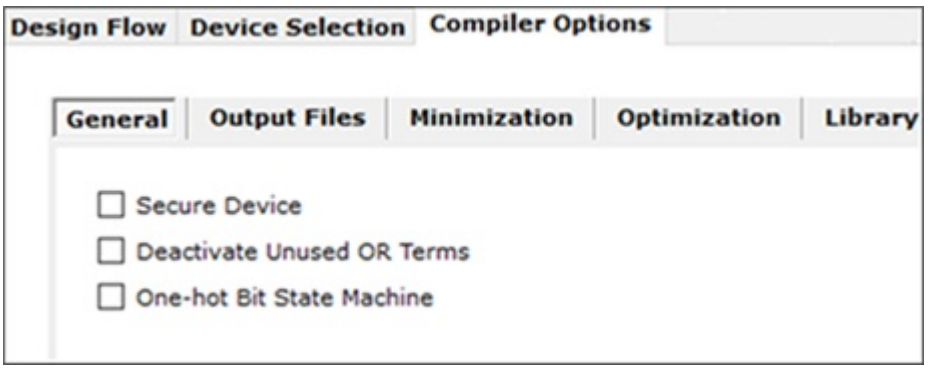
Note: The CUPL compiler and simulator require a carriage return to be present at the end of the CUPL design and simulation input files.

4.1.3. Select Options

Before compiling a design, users can set the compiler options and change the target device selection or device option settings in WinCUPL II.

In WinCUPL II, select the **Compiler Options** tab and then select the **General** tab. Ensure that all general compiler options are unchecked.

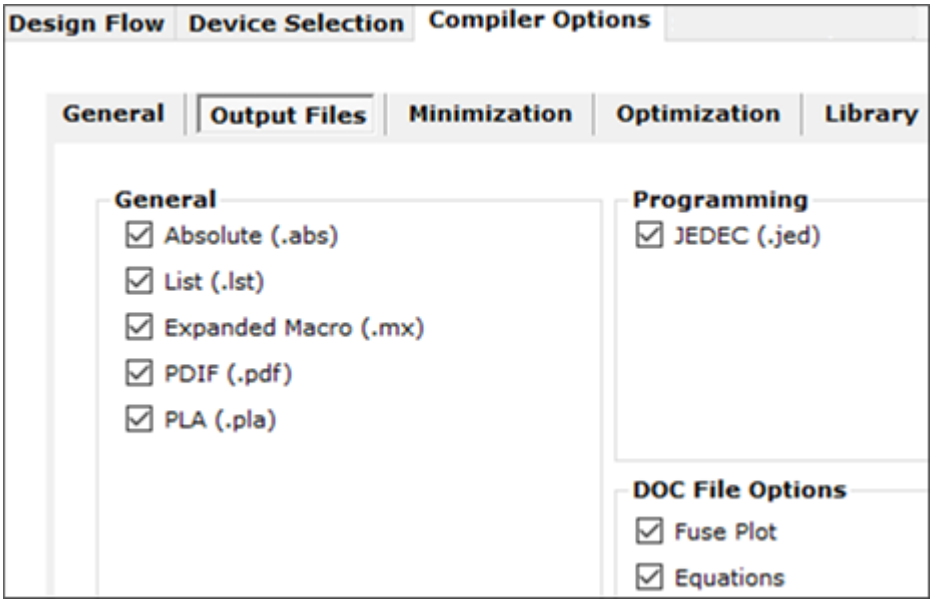
Figure 4-4. Compiler Options - General



Select the **Output Files** tab and check all options under the **General**, **Programming**, and **DOC File Options** sections.

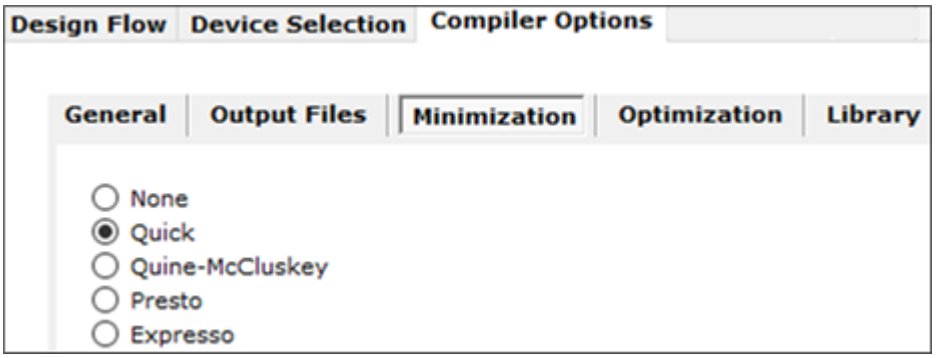
© 2026 Microchip Technology Inc. and its subsidiaries

Figure 4-5. Compiler Options – Output Files



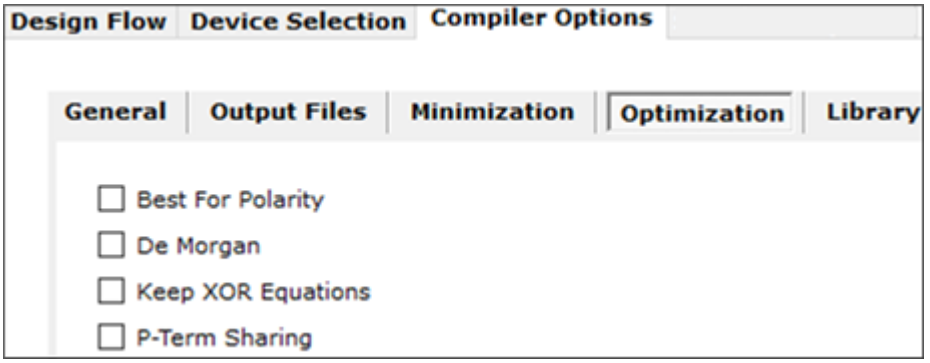
Select the **Minimization** tab and choose **Quick**.

Figure 4-6. Compiler Options – Minimization



Select the **Optimization** tab and uncheck all options.

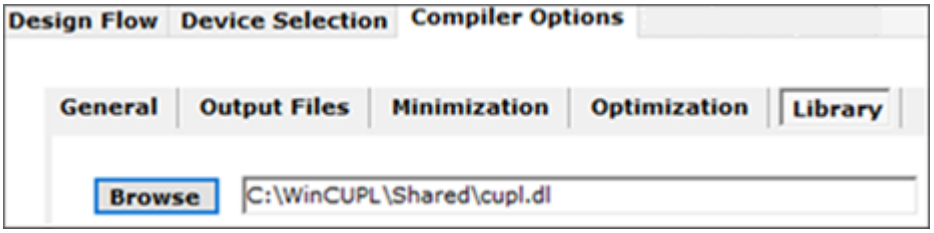
Figure 4-7. Compiler Options – Optimization



Select the **Library** tab and ensure that the correct library file (cupl.dl) is specified. The library file is located in the **Shared** folder under the WinCUPL II installation directory.

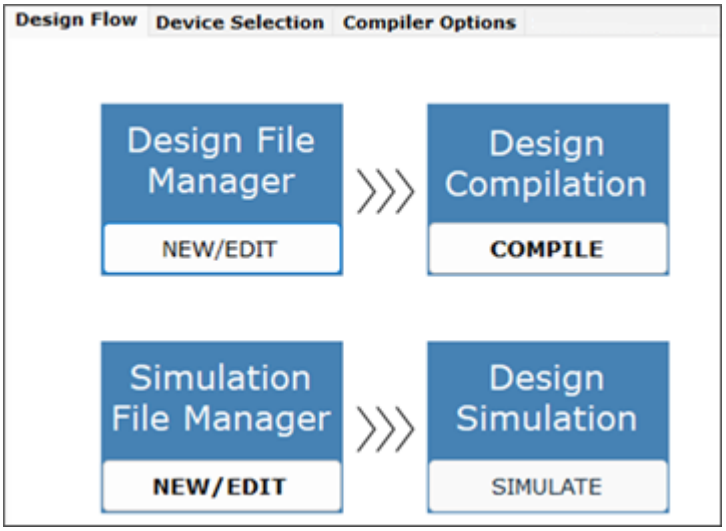
© 2026 Microchip Technology Inc. and its subsidiaries

Figure 4-8. Compiler Options – Library



In WinCUPL II, select the **Design Flow** tab to return to the design flow window.

Figure 4-9. Design Flow Tab



4.1.4. Compile Design

After creating the new CUPL design file (**AND_2.pld**) and setting up the compiler options, the next step is to compile the newly created CUPL design.

In WinCUPL II, under the **Design Flow** tab, click the **COMPILE** button under **Design Compilation** to compile the CUPL design.

Figure 4-10. Design Compilation

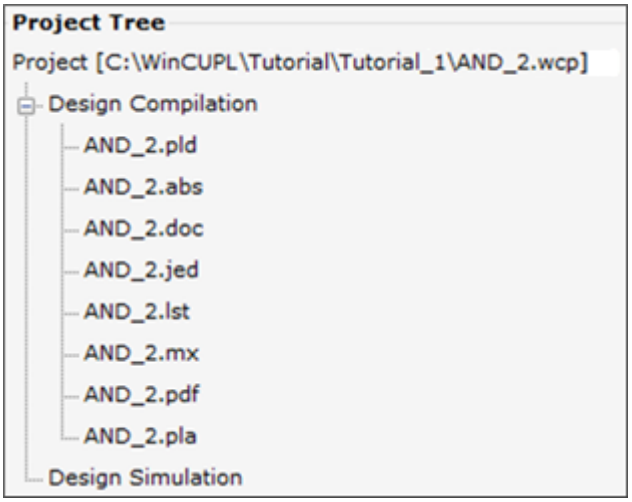


At the end of the compilation process, a message prompt indicating that the design has been successfully compiled will appear. This message is also shown in the **Message** section of the WinCUPL II window, as well as in the LOG file.

After successfully compiling a CUPL design, all the files generated during the compilation process will appear under the **Project Tree** section of the WinCUPL II window. You can open these files in the built-in text editor by double clicking a file in the Project Tree.

© 2026 Microchip Technology Inc. and its subsidiaries

Figure 4-11. Project Tree After Compilation



4.1.5. Create New CUPL Simulation Input File

The next step in the design development process is to create a new CUPL Simulation Input (.SI) file using the built-in text editor in WinCUPL II. This file is used by the CUPL simulator to run a functional simulation of the CUPL design.

In WinCUPL II, select the **Design Flow** tab and then click the **NEW/EDIT** button under **Simulation File Manager** to launch the text editor. The software will automatically create a new CUPL Simulation Input file named **AND_2.si**, based on the file name used for the project file.

Figure 4-12. Simulation File Manager



Enter the following text into the text editor:

© 2026 Microchip Technology Inc. and its subsidiaries

```
Name            AND_2;
PartNo          ;
Date            ;
Revision        ;
Designer        ;
Company         ;
Assembly        ;
Location        ;
Device          ;

ORDER: A, B, Q;

VECTORS:
00L
01L
10L
11H
00L
01L
10L
11H
00L
01L
10L
```

Note: The CUPL compiler and simulator require a carriage return to be present at the end of the CUPL design and simulation input files.

In the text editor, select **File → Save** to save the CUPL Simulation Input file. This file will be saved as **AND_2.si** in the **AND_2** project directory (for example, \WinCUPL\Tutorial\Tutorial_1).

4.1.6. Simulate Design

After creating the new CUPL Simulation input file (**AND_2.si**), the next step is to run a functional simulation of the CUPL design in WinCUPL II.

In WinCUPL II, under the **Design Flow** tab, click the **SIMULATE** button under **Design Simulation** to simulate the CUPL design.

Figure 4-13. Design Simulation

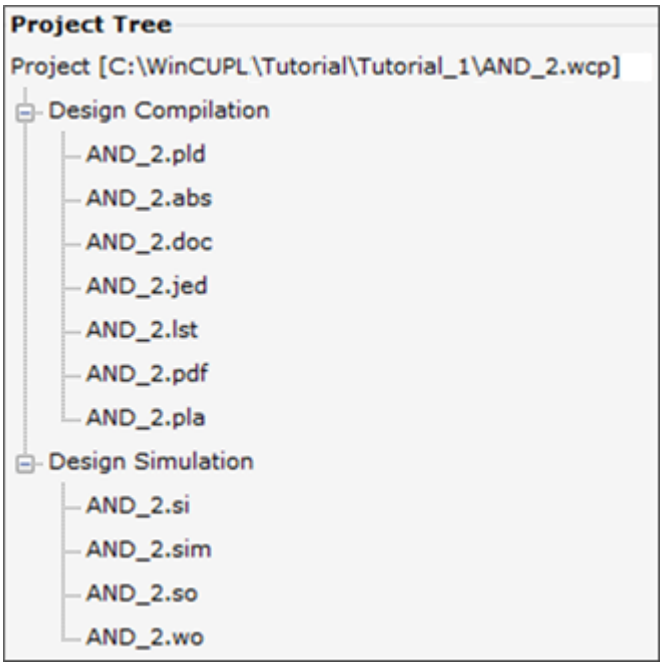


At the end of the simulation process, a message prompt indicating that the design has been successfully simulated will appear. This message is also shown in the **Message** pane of the WinCUPL II window, as well as in the LOG file.

© 2026 Microchip Technology Inc. and its subsidiaries

After simulating a CUPL design, all the files generated during the simulation process will appear under the **Project Tree** section of the WinCUPL II window. You can open these files in the built-in text editor by double clicking a file in the Project Tree.

Figure 4-14. Project Tree After Simulation



The simulation results are contained in the Simulation Output file (**AND_2.so**). You can double click this file in the Project Tree to open it in the built-in text editor and view the simulation results.

```
============
     ABQ
============
0001: 00L
0002: 01L
0003: 10L
0004: 11H
0005: 00L
0006: 01L
0007: 10L
0008: 11H
0009: 00L
0010: 01L
0011: 10L
```

If there are any simulation errors, they will be shown in this file.

After successfully compiling and simulating the **AND_2** design in WinCUPL II, select **File → Save Project** or click the **Save Project** shortcut button to save the project.

© 2026 Microchip Technology Inc. and its subsidiaries

5. Revision History

Revision A (May 2026)

Initial release of this document.

© 2026 Microchip Technology Inc. and its subsidiaries

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at https://www.microchip.com/en-us/about/legal-information/microchip-trademarks.

ISBN: 979-8-3371-3098-9

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP “AS IS”. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP’S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer’s risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip products are strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.

© 2026 Microchip Technology Inc. and its subsidiaries

Product Page Links

ATF1500A, ATF1500AL, ATF1502AS, ATF1502ASL, ATF1502ASV, ATF1504AS, ATF1504ASL, ATF1504ASV, ATF1504ASVL, ATF1508AS, ATF1508ASL, ATF1508ASV, ATF1508ASVL, ATF16LV8C, ATF16V8B, ATF16V8BQL, ATF16V8C, ATF16V8CZ, ATF22LV10C, ATF22LV10CQZ, ATF22V10B, ATF22V10C, ATF22V10CQZ, ATF2500C, ATF750C, ATF750CL, ATF750LVC, ATF22V10CQ

© 2026 Microchip Technology Inc. and its subsidiaries